

Database systems 1

dr hab. inż. Przemysław Korytkowski, prof. ZUT

Lecture 8 – Transactions and concurrency control

1

1

Transaction

Collections of operations that form a single logical unit of work are called **transactions**.

- The transaction consists of all operations executed between the **begin transaction** and **end transaction**.
- Usually, a transaction is initiated by a user program written in SQL or programming language (e.g., C++ or Java), with embedded database accesses in JDBC or ODBC.
- This collection of steps must appear to the user as a single, **indivisible** unit.
- Transaction either executes in its entirety or not at all.

2

Transactions

Transactions concerns:

- Single statement queries
- Multi statement queries

Examples:

- Changes to the database
- Money transfer
- Train/air ticket booking
- On-line payment

We need to be sure that the price is changed for all products.

```
START TRANSACTION;
UPDATE Product
SET Price = Price * 0,9
WHERE category = 'toys';
COMMIT;
```

```
START TRANSACTION;
UPDATE Bank SET amount = amount - 100
WHERE account = 4565434564;
UPDATE Bank SET amount = amount + 100
WHERE account = 8453476631;
INSERT INTO journal VALUES
(4565434564, 8453476631, 100, '22/05/11',...);
COMMIT;
```

3

Autocommit

Single statements are executed as transactions

```
START TRANSACTION;
UPDATE Product
SET Price = Price * 0,9
WHERE category = 'toys';
COMMIT;
```

=

```
UPDATE Product
SET Price = Price * 0,9
WHERE category = 'toys';
```

- Each SQL statement forms a single transaction on its own.
- Default mode in databases.

4

Protection against crashes/aborts

Client 1:
`INSERT INTO CheapProduct(name, price)`
`SELECT pname, price`
`FROM Product`
`WHERE price <= 0.99;`

Crash/abort!

`DELETE Product`
`WHERE price <=0.99;`

What goes wrong?

5

5

Protection against crashes/aborts

Client 1:
`START TRANSACTION;`
 1) `INSERT INTO CheapProduct(name, price)`
`SELECT pname, price`
`FROM Product`
`WHERE price <= 0.99;`
 2) `DELETE Product`
`WHERE price <=0.99;`
`COMMIT;`

Now we'd be fine!

6

6

Types of crashes/aborts

- A **computer failure** (system crash). A hardware, software, or network error occurs in the computer system during transaction execution. Hardware crashes are usually media failures—for example, main memory failure.
- **Transaction or system error.** Some operation in the transaction may cause it to fail, such as integer overflow or division by zero. Transaction failure may also occur because of erroneous parameter values or because of a logical programming error. Additionally, the user may interrupt the transaction during its execution.
- **Local errors or exception conditions detected by the transaction.** During transaction execution, certain conditions may occur that necessitate cancellation of the transaction. For example, data for the transaction may not be found. An exception condition, such as insufficient account balance in a banking database, may cause a transaction, such as a fund withdrawal, to be canceled.

7

Types of crashes/aborts

- **Disk failure.** Some disk blocks may lose their data because of a read or write malfunction or because of a disk read/write head crash. This may happen during a read or a write operation of the transaction.
- **Physical problems and catastrophes.** Problems that includes power or air-conditioning failure, fire, theft, sabotage, overwriting disks by mistake.

8

Concurrency – multiple users

```
Client 1: UPDATE Product
          SET Price = Price - 1.99
          WHERE category = 'toys';

Client 2: UPDATE Product
          SET Price = Price * 0.95
          WHERE category = 'toys';
```

Two managers attempt to discount products *concurrently*-
What could go wrong?

9

9

Concurrency – multiple users

```
Client 1: START TRANSACTION;
          UPDATE Product
          SET Price = Price - 1.99
          WHERE category = 'toys';
          COMMIT;

Client 2: START TRANSACTION;
          UPDATE Product
          SET Price = Price * 0.95
          WHERE category = 'toys';
          COMMIT;
```

Now results are predictable!

10

10

Reasons for concurrency

- **Improved throughput and resource utilization:**

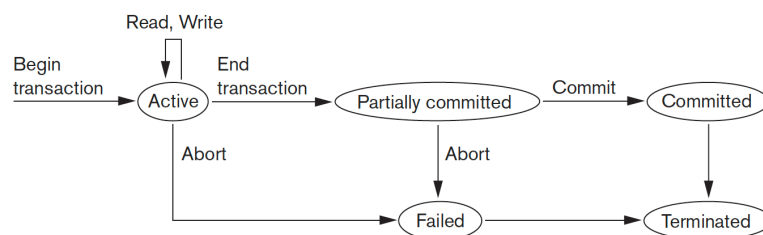
- The parallelism of the CPU and the I/O system allows to run multiple transactions in parallel.
- While a read or write on behalf of one transaction is in progress on one disk, another transaction can be running in the CPU, while another disk may be executing a read or write on behalf of a third transaction.
- It increases the throughput of the system. The processor and disk spend less time idle.

- **Reduced waiting time:**

- If transactions run serially, a short transaction may have to wait for a preceding long.
- If the transactions are operating on different parts of the database, it is better to let them run concurrently, sharing the CPU cycles and disk accesses among them.
- Concurrent execution reduces the unpredictable delays in running transactions.
- It reduces the average response time.

11

Transaction states



- **Active**, the initial state; the transaction stays in this state while it is executing.
- **Partially committed**, after the final statement has been executed.
- **Failed**, after the discovery that normal execution can no longer proceed.
- **Aborted**, after the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction.
- **Committed**, after successful completion.

12

Desirable properties of transactions – ACID

- **Atomicity.** Either all operations of the transaction are reflected properly in the database, or none are.
- **Consistency.** Execution of a transaction in isolation (i.e., with no other transaction executing concurrently) preserves the consistency of the database.
- **Isolation.** Even though multiple transactions may execute concurrently, the system guarantees that, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started or T_j started execution after T_i finished. Thus, each transaction is unaware of other transactions executing concurrently in the system.
- **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

13

Transaction manager

Buffer manager

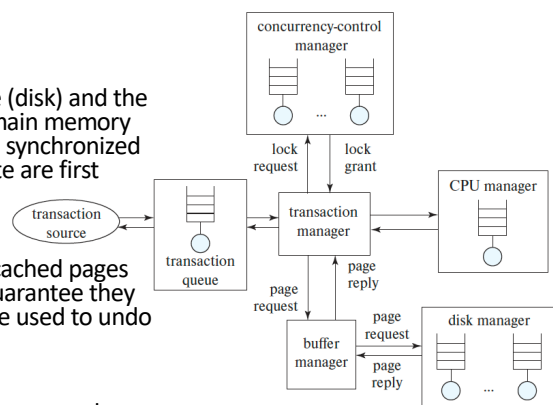
- Serves as an intermediary between persistent storage (disk) and the rest of the storage engine. It stages state changes in main memory and serves as a cache for the pages that haven't been synchronized with persistent storage. All changes to a database state are first applied to the cached pages.

Log manager

- Holds a history of operations (log entries) applied to cached pages but not yet synchronized with persistent storage to guarantee they won't be lost in case of a crash. Log entries can also be used to undo changes done by the aborted transactions.

Lock manager

- Guards access to resources and prevents concurrent accesses that would violate data integrity. Whenever a lock is requested, the lock manager checks if it is already held by any other transaction in shared or exclusive mode, and grants access to it if the requested access level results in no contradiction.



source: Silberschatz A., Korth, HF., Sudarshan, S. (2020) Database System Concepts, 7th ed., McGraw Hill.

14

Log manager

- Also called DBMS journal
- Provides **atomicity** and **durability** (ACID properties)
- An append-only disk structure is used for crash and transaction recovery.
- The page cache allows buffering changes to page contents in memory. Until the cached contents are flushed back to disk, the only disk-resident copy preserving the operation history is stored in the log.

If the database has crashed, all data that was not flushed is lost

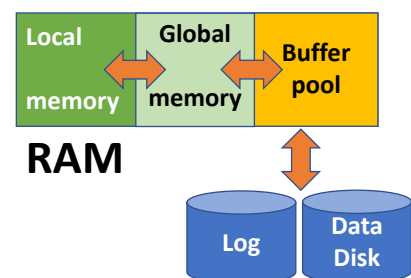
15

Log Manager

- The log consists of records. Every record has a unique, monotonically increasing log sequence number (LSN): internal counter or a timestamp.
- Since log records do not necessarily occupy an entire disk block, their contents are cached in the log buffer and are flushed on disk in a force operation.

Log types:

- UNDO <LSN, element, old values>
 - REDO <LSN, element, new values>
- or
- UNDO/REDO <LSN, element, old values, new values>



16

16

Write-Ahead Logging (WAL)

- Before a block of data in the main memory can be output to the database (in nonvolatile storage), all log records pertaining to data in that block must have been output to stable storage.
- Before the **<T_i commit>** log record can be output to stable storage, all log records pertaining to transaction T_i must have been output to stable storage.
- Transaction T_i enters the commit state after the **<T_i commit>** log record has been output to stable storage.
- Holds records indicating:
 - Operation that changes state of a database: INSERT, UPDATE, DELETE, CREATE, DROP,...
 - Transaction completion.

17

Write-Ahead Logging (WAL) – Checkpoints

When a system crash occurs, we must search the log to determine those transactions that need to be redone and those that need to be undone.

Checkpoints are added to the log to set the point beyond which search is not required:

- No updates are performed while the checkpoint operation is in progress.
- All modified buffer blocks are written to disk when the checkpoint is performed.

18

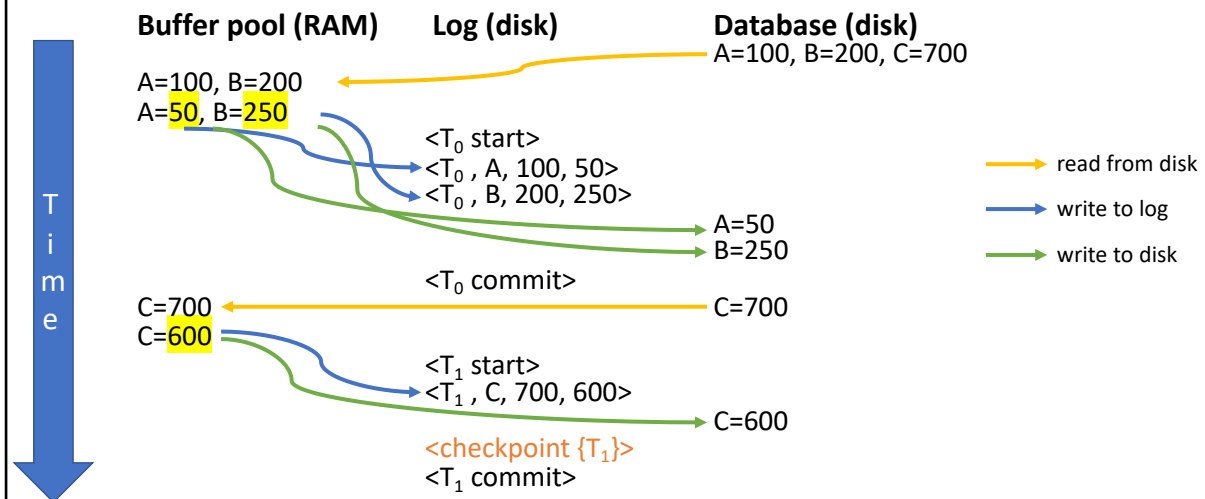
Write-Ahead Logging (WAL)

START TRANSACTION;

UPDATE Account **SET** amount = amount - 50 **WHERE** account = A;

UPDATE Account **SET** amount = amount + 50 **WHERE** account = B;

COMMIT;



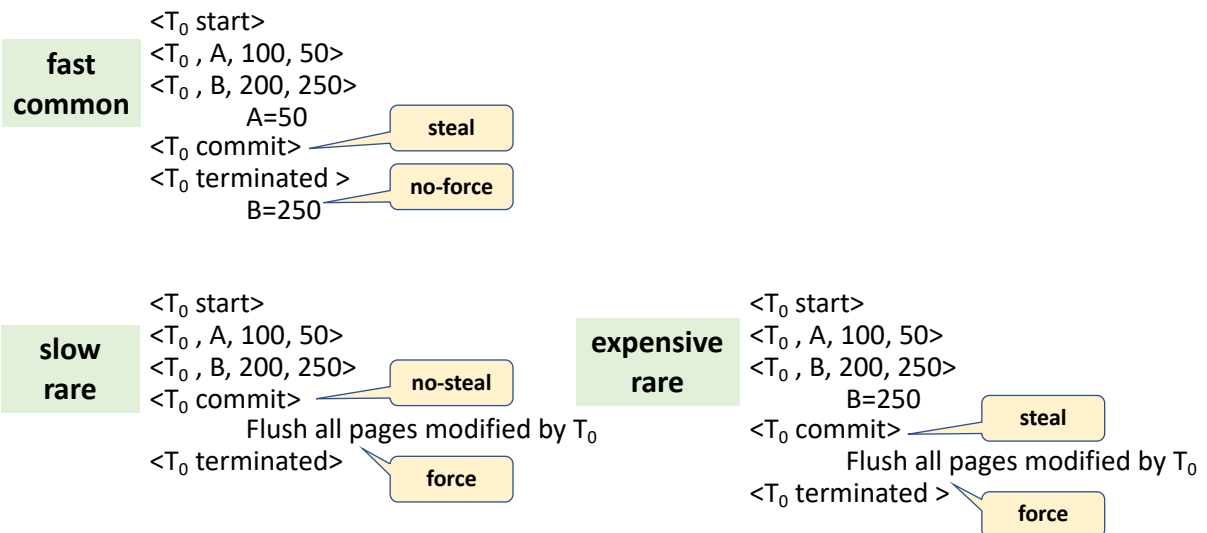
19

Log manager

- Flushing policies:
 - steal** – modified (dirty) pages may be written to disk before COMMIT.
 - no-steal** – modified (dirty) pages cannot be written to disk before COMMIT.
 - force** – all pages modified by the transaction are flushed at COMMIT.
 - no-force** – pages may remain dirty after COMMIT.
- STEAL + NO-FORCE** (most common, best performance, small buffer pool)
 - PostgreSQL, Oracle, InnoDB
 - Dirty pages can be flushed anytime
 - Commit does NOT force pages
 - UNDO + REDO required, ARIES recovery algorithm

20

Flushing policies



21

Recovery strategies

- extensive damage (for ex. disk crash)
 - the recovery method restores a past copy of the database that was backed up to archival storage and reconstructs a more current state by reapplying or redoing the operations of committed transactions from the backed-up log, up to the time of failure.
- database on disk is not physically damaged
 - the recovery strategy is to identify any changes that may cause an inconsistency in the database. It may be necessary to redo some operations in order to restore a consistent state of the database.
 - The recovery protocol does not need a complete archival copy of the database. Rather, the entries kept in the online system log on disk are analyzed to determine the appropriate actions for recovery.

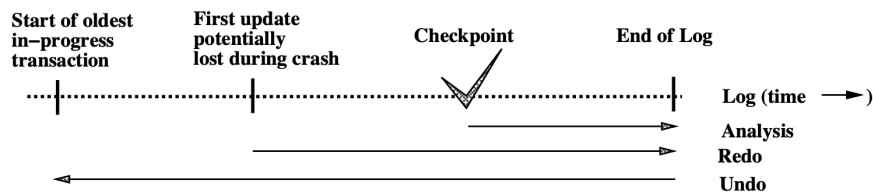
22

ARIES Recovery Algorithm

ARIES is a recovery algorithm that is designed to work with a **steal, no-force** policy.

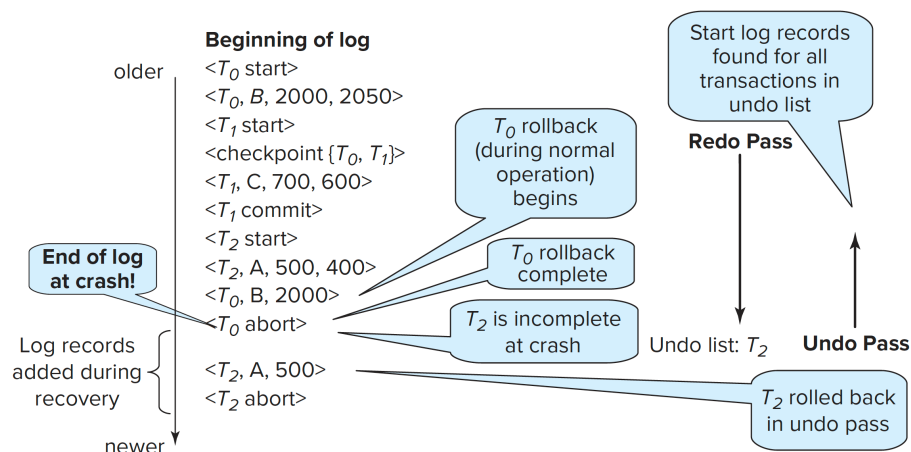
After a crash the recovery manager is invoked, it proceeds in three phases:

1. In the **Analysis phase**, it identifies dirty pages in the buffer pool (changes that have not been written to disk) and active transactions at the time of the crash.
2. In the **Redo phase**, it repeats all actions, starting from an appropriate point in the log, and restores the database state to what it was at the time of the crash.
3. Finally, in the **Undo phase**, it undoes the actions of transactions that did not commit, so that the database reflects only the actions of committed transactions.



23

Logged actions and actions during recovery



source: Silberschatz A., Korth, HF., Sudarshan, S. (2020) Database System Concepts, 7th ed., McGraw Hill.

24

Concurrency Control

Concurrency control is a set of techniques for handling interactions between concurrently executing transactions.

- Pessimistic (conservative) concurrency control (PCC)
 - Lock-based and nonlocking conservative methods.
 - Can result in a deadlock when multiple transactions wait for each other to release a lock in order to proceed.
- Multiversion concurrency control (MVCC)
 - Guarantees a consistent view of the database at some point in the past identified by the timestamp by allowing multiple timestamped versions of the record to be present.
- Optimistic concurrency control (OCC)
 - Transactions do not block each other, maintain histories of their operations, and check these histories for possible conflicts before commit. If execution results in a conflict, one of the conflicting transactions is aborted.

F
A
S
T
E
R

R
E
L
I
A
B
L
E

25

Serializability

- Transactions consist of **read** and **write** operations executed against the database state, and business logic (transformations, applied to the read contents).
- A **schedule** is a list of operations required to execute a set of transactions from the database-system perspective (INSERT, UPDATE, DELETE, COMMIT), all other operations are assumed to be side-effect free.
- A **complete schedule** contains all operations from every transaction executed.
- **Correct schedules** are logical equivalents to the original lists of operations, but their parts can be executed in parallel or get reordered for optimization purposes, if this does not violate ACID properties and the correctness of the results of individual transactions.

26

Serializability

- A schedule is **serial** when transactions in it are executed completely independently and without any interleaving: every preceding transaction is fully executed before the next one starts.



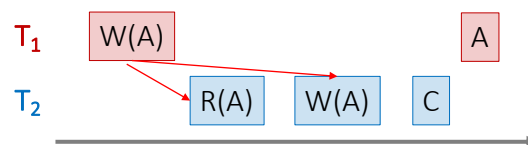
- A schedule is **serializable** if it is equivalent to some complete serial schedule over the same set of transactions. It produces the same result as if we executed a set of transactions one after another in some order.



27

Read and write anomalies

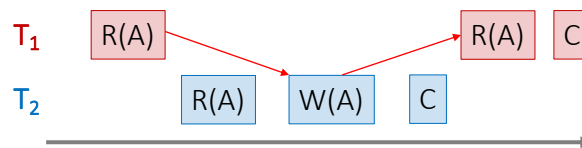
- A **dirty read** is a situation in which a transaction can read uncommitted changes from other transactions.
- W-R conflict



28

Read and write anomalies

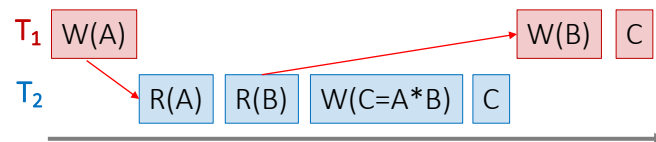
- A **nonrepeatable read** is a situation in which a transaction queries the same row twice and gets different results.
- R-W conflict



29

Read and write anomalies

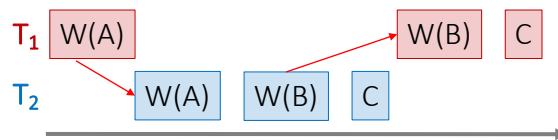
- A **phantom read** is when a transaction queries the same set of rows twice and receives different results. It is similar to a nonrepeatable read but holds for range queries.
- W-R conflict



30

Read and write anomalies

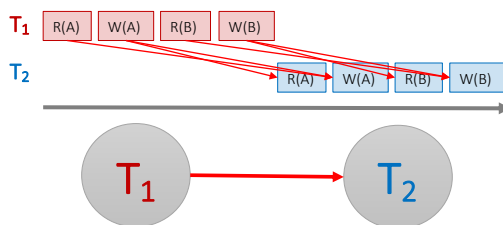
- A **lost update** occurs when two transactions attempt to update the value. If both are allowed to commit, the results of one of them will be overwritten by the results of other, and the update from first one will be lost.
- W-W conflict



31

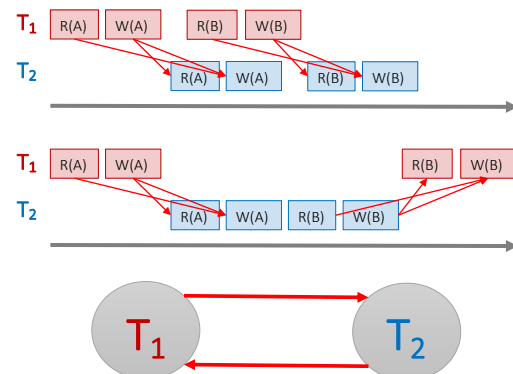
The Conflict Graph

Serial Schedule:



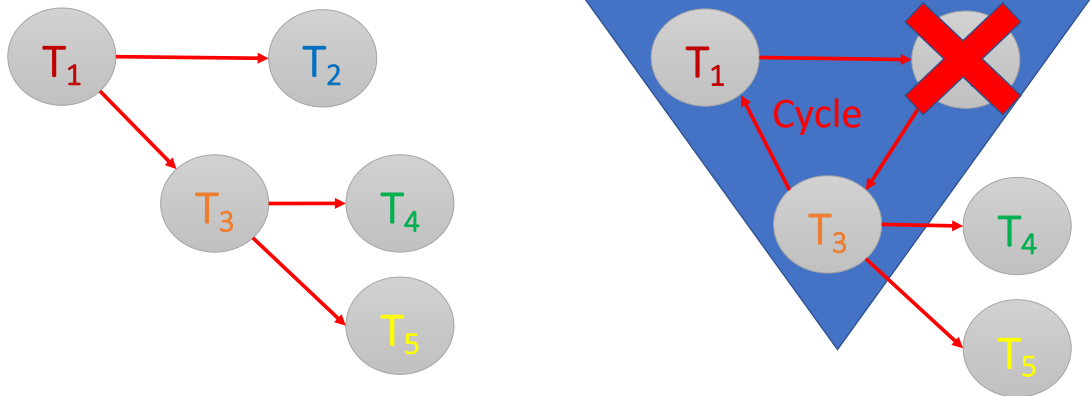
Schedule is **conflict serializable** if and only if its conflict graph is **acyclic**

Interleaved Schedules:



32

The Conflict Graph



Schedule is **conflict serializable** if and only if its conflict graph is acyclic

33

Optimistic Concurrency Control

- Optimistic concurrency control assumes that transaction conflicts occur rarely and, instead of using locks and blocking transaction execution.
- **Read phase**
 - The transaction executes its steps in its own private context, without making any of the changes visible to other transactions. After this step, all transaction dependencies (read set) are known, as well as the side effects the transaction produces (write set).
- **Validation phase**
 - Read and write sets of concurrent transactions are checked for the presence of possible conflicts between their operations that might violate serializability. If some of the data the transaction was reading is now out-of-date, or it would overwrite some of the values written by transactions that were committed during its read phase, its private context is cleared, and the read phase is restarted. The validation phase determines whether or not committing the transaction preserves ACID properties.
- **Write phase**
 - If the validation phase hasn't determined any conflicts, the transaction can commit its write set from the private context to the database state.

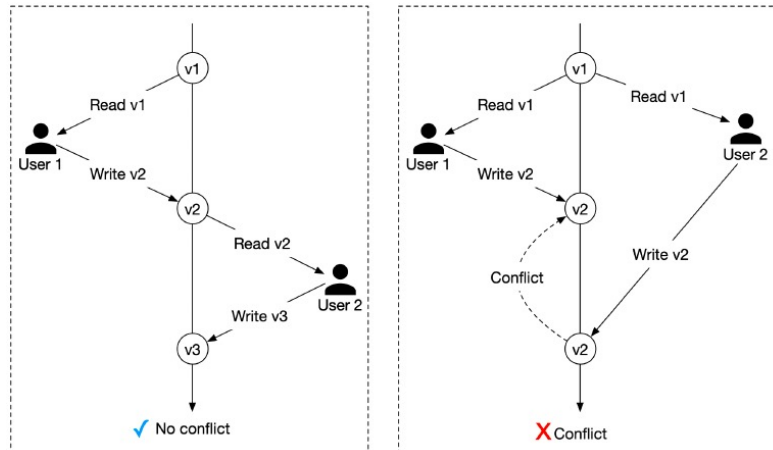
34

Optimistic Concurrency Control

Two ways to implement:

- version number
- timestamp

Version number is generally considered to be a better option because the server clock can be inaccurate over time.



35

Multi-Version Concurrency Control (MVCC)

- A transaction running in **SNAPSHOT ISOLATION** mode operates on a version of the database as it existed at the time the transaction began; subsequent updates by other transactions are invisible to the transaction.
- When the transaction starts, it gets a unique start-timestamp;
- when it commits it gets a unique end-timestamp.
- The transaction commits only if no other transaction with an overlapping start/end-transaction pair wrote data that this transaction also wrote.
- Multi-version concurrency can be implemented using locking, scheduling, and conflict resolution techniques (such as two-phase locking), or timestamp ordering.

36

36

Pessimistic Concurrency Control

- These schemes determine transaction conflicts while they're running and block or abort their execution.
- **Timestamp ordering**
 - Each transaction has a timestamp. Whether or not transaction operations are allowed to be executed is determined by whether or not any transaction with an earlier timestamp has already been committed.
 - To implement that, the transaction manager has to maintain `max_read_timestamp` and `max_write_timestamp` per value, describing read and write operations executed by concurrent transactions.
- **Lock-Based Concurrency Control**
 - uses explicit locks on the database objects rather than resolving schedules, like protocols such as timestamp ordering do.
 - One of the most widespread lock-based techniques is two-phase locking (2PL)

37

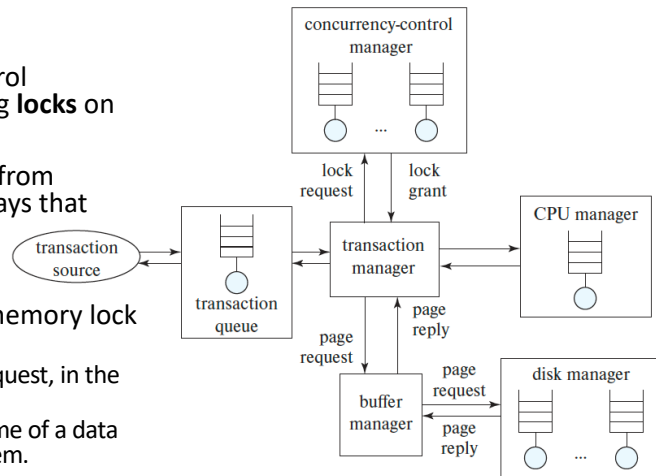
Locks

- A transaction can acquire a **lock on the entire database** before it starts and releases the lock after it has committed. While a transaction holds a lock, no other transaction is allowed to acquire the lock, and all must therefore wait for the lock to be released. As a result of the locking policy, only one transaction can execute at a time. Therefore, only serial schedules are generated.
- Instead of locking the entire database, a transaction could instead **lock only those data items that it accesses**. Under such a policy, the transaction must hold locks long enough to ensure serializability, but for a period short enough not to harm performance excessively.

38

Locks

- A typical scheduler (concurrency-control manager) does its work by maintaining **locks** on certain pieces of the database.
- These **locks** prevent two transactions from accessing the same piece of data in ways that interact badly.
- **Locks** are generally stored in a main-memory lock table:
 - A linked list of records, one for each request, in the order in which the requests arrived.
 - It uses a hash table, indexed on the name of a data item, to find the linked list for a data item.



39

Lock modes

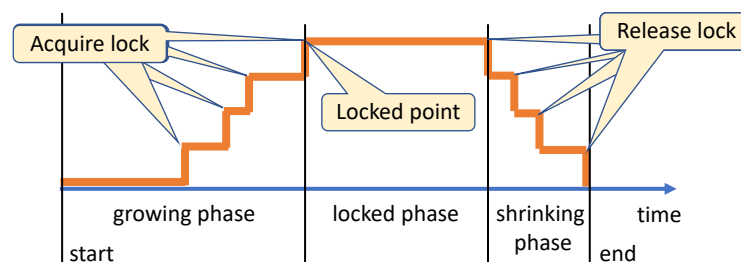
- **X (exclusive) lock** on object before **writing**.
 - If a transaction holds, no other transaction can get a lock (S or X) on that object.
- **S (shared) lock** on object before **reading**
 - If a transaction holds, no other transaction can get an X lock on that object
- **U (update) lock** on object that are planned **reading** and then **writing**

		Lock requested		
		S	X	U
Lock held in mode	S	Yes	No	Yes
	X	No	No	No
	U	No	No	No

40

Two-phase locking (2PL)

- The **growing phase** (also called the expanding phase), during which all locks required by the transaction are acquired and no locks are released.
- The **shrinking phase**, during which all locks acquired during the growing phase are released.
- 2PL ensures acyclicity of the precedence graph and therefore allows only **conflict serializable schedules**.



41

Locks – summary

- Locking is generally considered to have a **high overhead**, because every read or write operation is preceded by a system locking request.
- The use of locks can also cause two additional problems:
 - Deadlock - a system is in a deadlock state if there exists a set of transactions such that every transaction in the set is waiting for another transaction in the set.
 - Starvation – occurs when a transaction cannot proceed for an indefinite period of time while other transactions in the system continue normally. This may occur if the waiting scheme for locked items is unfair in that it gives priority to some transactions over others.

42

Deadlock prevention

- **Ordering Resources:** requiring transactions to acquire resources according to some lexicographic order of the resources will prevent a deadlock from arising.
- **Timestamp-Based:** maintain a timestamp and base their abort/wait decision on whether the requesting transaction is newer or older than the one with the resource it wants.
 - **wait-die scheme**, an older requesting transaction waits, and a newer one is rolled back with the same timestamp.
 - **the wound-wait scheme**, a newer transaction waits and an older one forces the transaction with the resource to roll back and give up the resource.

43

43

Explicit (manual) Locks

```
LOCK TABLES students READ, courses WRITE;  
SELECT * FROM students WHERE SID = 1;  
UPDATE courses SET course = 'math' WHERE course = 'mathematics';  
UNLOCK TABLES;
```

44

Isolation levels (SQL-92)

An isolation level specifies how and when parts of the transaction can and should become visible to other transactions.

Level	Lost update	Dirty read	Unrepeatable read	Phantoms	READ	WRITE
READ UNCOMMITTED	Not possible	Possible	Possible	Possible	No Lock	X Lock
READ COMMITTED	Not possible	Not possible	Possible	Possible	MVCC	X Lock
REPEATABLE READ	Not possible	Not possible	Not possible	Possible	MVCC	X Lock
SERIALIZABLE	Not possible	Not possible	Not possible	Not possible	S Lock	X Lock

Achieving isolation comes at a cost: to prevent incomplete or temporary writes from propagating over transaction boundaries, we need additional coordination and synchronization, which negatively impacts the performance.

45

45

Isolation levels

```
SET GLOBAL TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
SELECT @@tx_isolation;
```

Default in MariaDB:
REPEATABLE READ

46

Concurrency control in MySQL

- The InnoDB transaction model aims to combine the best properties of a **multi-versioning database** with traditional **two-phase locking**.
- InnoDB performs locking at the row level and runs queries as nonlocking consistent reads by default, in the style of Oracle.
- The lock information in InnoDB is stored space-efficiently so that lock escalation is not needed.
- Typically, several users are permitted to lock every row in InnoDB tables, or any random subset of the rows, without causing InnoDB memory exhaustion.

Source: <https://dev.mysql.com/doc/refman/8.0/en/innodb-transaction-model.html>

47

TPC-C benchmark – www.tpc.org/

- Transaction Processing Performance Council
- TPC Benchmark C is an on-line transaction processing (OLTP) benchmark. TPC-C involves a mix of five concurrent transactions of different types and complexity either executed on-line or queued for deferred execution. The database is comprised of nine types of tables with a wide range of record and population sizes.

48

TPC-C benchmark – www.tpc.org/

Source: Michael Stonebraker, Samuel Madden, Daniel J. Abadi, Stavros Harizopoulos, Nabil Hachem, and Pat Helland. 2007. The end of an architectural era: (it's time for a complete rewrite). In Proceedings of the 33rd international conference on Very large data bases (VLDB '07). VLDB Endowment, 1150–1160.

- Test computer: A dual-core 2.8GHz CPU computer system, with 4 Gbytes of main memory and four 250 GB SATA disk drives.
- **850 transactions per second.**
- The bottleneck was logging overhead. The system spent about 2/3 of its total elapsed time inside the logging system.
- If logging was turned off completely, then throughput would increase to about **2,500 transactions per second.**

49

TPC-E benchmark

Characteristic	TPC-E
Tables	33
Columns	188
Min Cols / Table	2
Max Cols / Table	24
Data Type Count	Many
Data Types	UID, CHAR, NUM, DATE, BOOL, LOB
Primary Keys	33
Foreign Keys	50
Tables w/ Foreign Keys	27
Check Constraints	22
Referential Integrity	Yes

Result ID:	120111901
Status:	Accepted Result
Report Date:	11/19/20
TPC-E Rev:	1.14.0
TPC Pricing Spec:	2.6.0
Business Recovery Time:	00:31:53
Auditor:	Doug Johnson
System Information	
Total System Cost:	1,033,292.00 USD
TPC-E Throughput:	12,162.59 transactions per second
Price/Performance:	12,162.59
TPC-Energy Metric	Not reported
Availability Date:	11/19/20
Database Manager:	Microsoft SQL Server 2019 Enterprise Edition
Operating System:	Microsoft Windows Server 2016 Standard Edition
Initial Database Size:	53,385 (GB)
Memory Size:	3,072 (GB)
Spindle Count:	142
Redundancy Level:	1
For Redundancy Level Details Click Here	
CPU Type:	Intel Xeon Platinum 8380H Processor 2.90GHz
Total # of Processors:	4
Total # of Cores:	112
Total # of Threads:	224
Cluster:	N
Client Specific Information	
# of Clients:	1
CPU Type:	Intel Xeon Gold 6248R Processor 3.00GHz
Total # of Processors:	2
Total # of Cores:	48
Total # of Threads:	96

50

Relational databases architectural features

RDBMSs were designed for the business data processing.

- Disk oriented storage and indexing structures
- Multithreading to hide latency
- Locking-based concurrency control mechanisms
- Log-based recovery

51

Thank you for your
attention!

Time for questions

52

52